

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Concept Mapping

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	Assessment:	Assessment Tool 1 – Concept Mapping
Weightage:	35% of CIA	Total Marks:	100
CO1 Statement:	<i>Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.</i>		
Student Name:	Shahana	Reg. No.:	192524479
Section / Batch:		Date:	10/07/2

Code of Conduct

I

_____Shahana(192524479)_____

__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __Shahana_____

Instructions

- This test contains 5 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should be with following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Linked data structure	Basic data structure	1	20
Primitive and Non-Primitive Data Structure - Linear and Non-Linear Data Structure	Classifications of Linear data structure	2	20
Search Data Structure	Linear Search and Binary Search	3	20
Recursion, Performance analysis	Recursion	4	20
Tree Data Structures	Classifications of Tree data structure	5	20
GRAND TOTAL:			100 Marks

Annexure A – Concept Mapping

1. A student attendance system records daily presence data. Construct a concept map linking Array → Fixed Size → Sequential Access → Linked List → Dynamic Allocation → Flexibility. Explain how each structure works and justify which is more suitable for dynamic data updates.
(20 Marks)
 2. A weather monitoring system collects temperature, humidity, and wind data continuously. Construct a concept map linking Primitive Data Types → Non-Primitive Structures → Linear Structures → Non-Linear Structures → Applications. Include examples and explain how each structure supports continuous data processing.
(20 Marks)
 3. A digital archive stores millions of documents. Construct a concept map linking Searching → Linear Search → Binary Search → Sorted Data → Time Complexity → Performance. Analyze the impact of data organization on search efficiency and justify the best technique.
(20 Marks)
 4. A program processes nested expressions using recursion and may encounter deep nesting. Construct a concept map linking Recursion → Call Stack → Stack Overflow → Error Handling → Efficiency. Analyze potential issues and justify strategies to handle them. (20 Marks)
 5. A company maintains hierarchical employee records with multiple levels of management. Construct a concept map connecting Tree → Nodes → Levels → Traversal (DFS/BFS) → Searching → Efficiency. Analyze how traversal techniques affect performance and justify the most suitable approach for retrieving employee data.
(20 Marks)
-

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Student Attendance System Data Structure Concept Map

Modify

Which Structure is More Suitable for Dynamic Attendance Updates

- Insertion, deletion and deletion linked list functions are complex and difficult because existing pointers of linked list are not aware of new elements
- Dynamic memory management is always a headache, either for changing attendance records and student information
- Scalability for queries as classes expand is difficult, linked list scales more naturally than arrays or fixed capacity
- Insertion/deletion cost is high for sequential access in the middle of array, other requires moving many elements until becomes expensive as the array grows
- Capacity issues arise when you require a listing of new students, if changes can be costly and disruptive
- Insertion/deletion are more complex when the number of records is stable and often increasing in the near future
- For dynamic data systems, linked lists are generally more suitable as they can support frequent additions and removals with less disruption of memory
- For fast flow, as much as a linked list, arrays can still be useful, especially when some records are stable
- For a non-world attendance system with frequent daily entries, some changes may be better suited to arrays, flexibility and system efficiency may be better in the long-term design choice

Array vs Linked List

- Array
 - Efficient for memory access, as it is stored sequentially in memory, which is faster than jumping between nodes
 - Index-based access, any element can be accessed directly using its position, making it faster than linked lists
 - Space efficiency, the size is usually chosen ahead of time to grow beyond capacity requires re-allocating and copying
 - Attendance records are often stable, records are fast access to new students, but space efficiency is not a priority
- Linked List
 - Nodes with pointers to next node, the pointer to next node is a variable, which can be changed
 - Dynamic memory allocation, nodes are created as needed, no need to pre-allocate space
 - Flexibility, linked list is more flexible, adding or removing nodes is easier, no need to shift elements
 - Attendance records, linked list is more suitable for records that change frequently, such as attendance records

Flexibility

- Definition
 - Flexibility: the ability of a structure to accommodate changes efficiently without major reorganization
 - Attendance records, the attendance database can handle changes in attendance records, such as adding or removing students, without major reorganization
- Application to daily attendance records
 - Non-world attendance system, changes are frequent, and the structure should support these changes efficiently
 - System resilience, attendance records are often updated, the structure should support frequent updates without major reorganization

Dynamic Allocation

- Definition
 - Dynamic allocation: memory is managed as needed, rather than pre-allocated in one fixed block
 - Attendance records, the attendance database can handle changes in attendance records, such as adding or removing students, without major reorganization
- Application to daily attendance records
 - Non-world attendance system, changes are frequent, and the structure should support these changes efficiently
 - System resilience, attendance records are often updated, the structure should support frequent updates without major reorganization

Array

- Definition
 - Array: a linear structure of contiguous elements in contiguous memory locations, with each element accessible by an index
 - Attendance records, the attendance database can handle changes in attendance records, such as adding or removing students, without major reorganization
- Application to daily attendance records
 - Non-world attendance system, changes are frequent, and the structure should support these changes efficiently
 - System resilience, attendance records are often updated, the structure should support frequent updates without major reorganization

Fixed Size

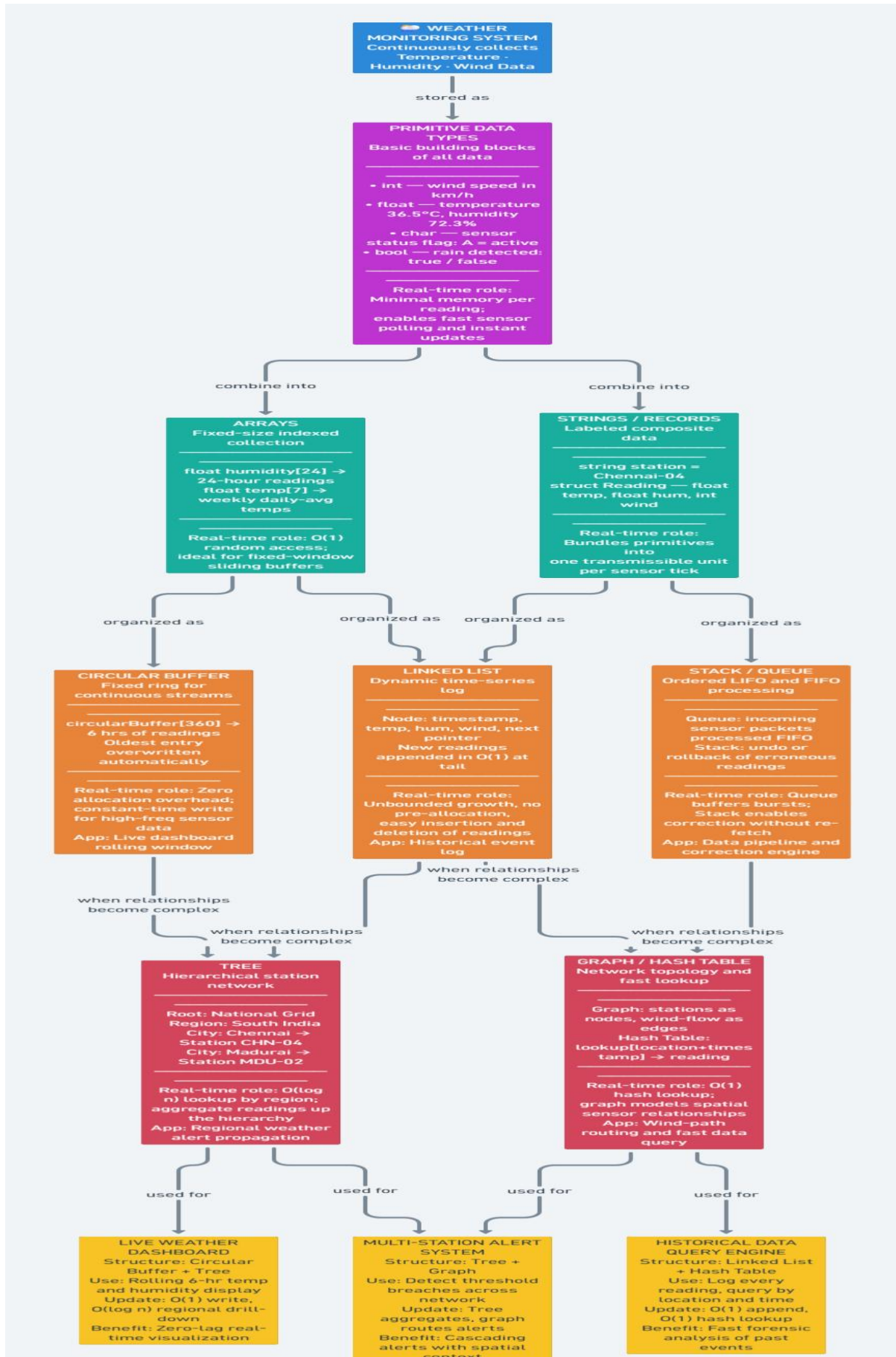
- Definition
 - Fixed size: the array has a set capacity that cannot be changed dynamically
 - Attendance records, the attendance database can handle changes in attendance records, such as adding or removing students, without major reorganization
- Application to daily attendance records
 - Non-world attendance system, changes are frequent, and the structure should support these changes efficiently
 - System resilience, attendance records are often updated, the structure should support frequent updates without major reorganization

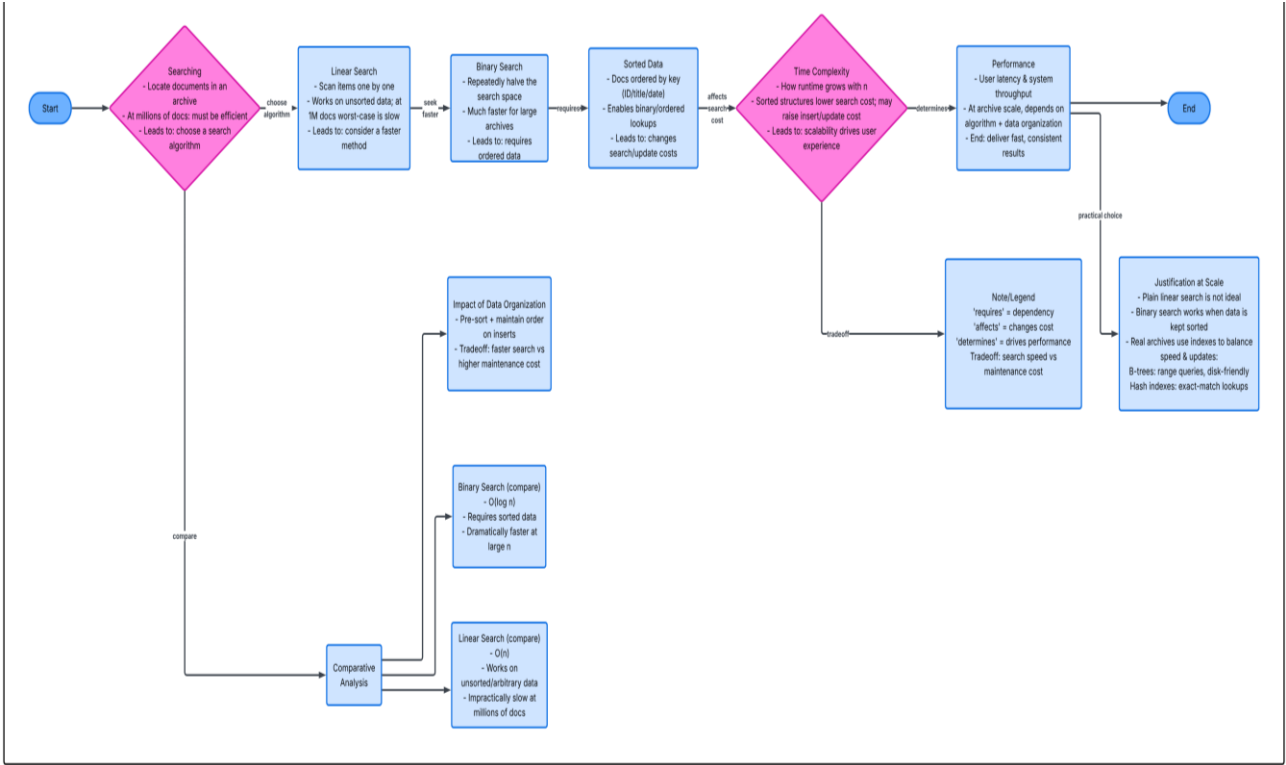
Sequential Access

- Definition
 - Sequential access: the structure allows access to elements in a sequential manner, typically through a pointer to the next element
 - Attendance records, the attendance database can handle changes in attendance records, such as adding or removing students, without major reorganization
- Application to daily attendance records
 - Non-world attendance system, changes are frequent, and the structure should support these changes efficiently
 - System resilience, attendance records are often updated, the structure should support frequent updates without major reorganization

Linked List

- Definition
 - Linked list: a linear structure made of nodes, where each node contains data and a pointer to the next node
 - Attendance records, the attendance database can handle changes in attendance records, such as adding or removing students, without major reorganization
- Application to daily attendance records
 - Non-world attendance system, changes are frequent, and the structure should support these changes efficiently
 - System resilience, attendance records are often updated, the structure should support frequent updates without major reorganization





TREE ★ Definition A hierarchical, non-linear data structure with a root node and sub-nodes connected by edges, with no cycles. 📁 In Company Records Models the entire company hierarchy. The CEO sits at the root. The tree represents all reporting relationships — from the executive suite down to individual contributors. → A tree is made up of individual units called Nodes — every person in the company is represented as one.	NODES ★ Definition The fundamental units of a tree. Each node holds data and references to its children (and optionally its parent). 📁 In Company Records Each employee is a node. CEO = root node (no parent). Department heads = Internal nodes (have both parent and children). Individual employees = leaf nodes (no children). → Nodes are organized by their distance from the root — this grouping defines Levels, which correspond directly to management tiers.	LEVELS ★ Definition A level groups all nodes that are the same number of edges from the root. Root = Level 0, its children = Level 1, and so on. 📁 In Company Records Level 0: CEO. Level 1: C-suite / VP's. Level 2: Directors. Level 3: Managers. Level 4+: Team Leads and individual contributors. Levels make org charts readable. → To process or access any level or node, the tree must be systematically visited — this requires Traversal.	**TRAVERSAL (DFS / BFS)** ★ Definition The process of visiting every node in the tree in a specific order. Two main strategies: Depth-First Search (DFS) and Breadth-First Search (BFS). 📁 In Company Records DFS: Follow one reporting chain all the way down before exploring another branch. BFS: Visit all Level-1 employees, then all Level-2, and so on — explores the entire org level by level. → The order in which nodes are visited determines how efficiently you can search for specific employees or groups.	SEARCHING ★ Definition The process of locating one or more specific nodes within the tree based on a query (e.g., by name, role, department, or level). 📁 In Company Records Find a specific employee's manager (DFS is efficient). Find all employees in one department at the same level (BFS is efficient). Tree balance and depth affect how quickly results are found. → The speed and scalability of searching depends on how efficiently the traversal is performed — this is measured as search Efficiency.	EFFICIENCY ★ Definition A measure of how well an algorithm uses time and space resources. Expressed via Big O notation ($O(n)$, $O(\log n)$, etc.). 📁 In Company Records Both DFS and BFS run in $O(n)$ time — every employee may need to be visited. Balanced trees reduce average search time. Space: BFS uses $O(w)$ memory (w = max width). DFS uses $O(d)$ memory (d = max depth).
--	--	---	--	---	---

Traversal Comparative Analysis

DFS (Depth-First Search) Strategy: Explore one branch fully before backtracking (uses a stack — explicit or via recursion). Best for: - Finding a specific employee's full reporting chain - Checking deep hierarchy paths (e.g., who reports to whom all the way up) - Detecting cycles (not relevant here, but useful in general) - Pre-order / in-order / post-order processing Time Complexity: $O(n)$ — visits all n nodes in worst case Space Complexity: $O(d)$ — d = maximum depth of tree. Efficient for wide, shallow trees; costly for deep trees. ✓ Verdict: Best when the query targets a deep, narrow path (e.g., reporting chain from CEO to a specific employee).	BFS (Breadth-First Search) Strategy: Explore nodes level by level using a queue. Best for: - Finding all employees at a specific management level - Shortest path between two employees in the hierarchy - Level-based org-wide reporting (e.g., all Directors) - Finding the closest (shallowest) matching employee Time Complexity: $O(n)$ — visits all n nodes in worst case Space Complexity: $O(w)$ — w = maximum width (breadth) of tree. Costly for wide orgs at a single level. ✓ Verdict: Best when the query targets a horizontal slice of the org (e.g., all Managers across all departments).
--	--

Searching, Efficiency & Traversal Strategy

Searching & Traversal Efficiency How traversal method affects search: - DFS is ideal for deep path queries (e.g., "Who does Alice report to, all the way to the CEO?"). It follows one branch completely, making it natural for lineage lookups. - BFS is ideal for horizontal queries (e.g., "List all employees at Level 3"). It processes every node at a given depth before going deeper. Impact of tree balance on performance: - Balanced tree → $O(\log n)$ average search depth, minimal traversal needed. - Skewed/deep tree (e.g., single chain of managers) → DFS degrades to $O(n)$ in worst case. - Wide tree (many employees per manager) → BFS uses $O(w)$ space, where w can be very large at lower levels. Key queries and recommended approach: - Find all reports under CTO: DFS from CTO node - Find all Level-3 Managers org-wide: BFS from root, stop at depth 3 - Shortest path between two employees: BFS (guarantees shortest path in unweighted trees)	Final Recommendation: Hybrid Strategy Reporting Chain Queries Use DFS — navigate straight down one branch (e.g., CEO → VP → Director → Manager → Employee). DFS naturally follows this path with minimal overhead. Department-Wide Lookups Use BFS — scan all nodes at a given level or within a subtree rooted at a department head. BFS processes all children before going deeper. Level-Based Reporting Use BFS — e.g., List all Level 3 Managers. BFS visits level by level, so reaching Level 3 requires only 3 queue iterations rather than full traversal. Hybrid Approach (Recommended) In production systems, use a hybrid strategy. BFS for level/department-wide queries and DFS for deep path queries. Index frequently queried attributes (level, department, manager_id) in a database to reduce full traversal needs. Tree balance should be monitored — highly imbalanced orgs (very deep chains) degrade DFS performance. Summary: Time/Space Trade-offs Both DFS & BFS: $O(n)$ time complexity DFS space: $O(d)$ — depth of tree BFS space: $O(w)$ — width of tree For real-world org data: use DB indexes + BFS/DFS hybrid at query time
---	---